# DLDCA Lab 3: Verilog

Week 4

August 21, 2026

## Introduction

This week we continue our exploration of Verilog by designing larger, more realistic systems. We will practice breaking a problem into smaller, well-defined components and wiring them together – a core skill in hardware design.

We will write **Finite State Machines (FSMs)**: circuits that maintain state and transition between well-defined modes of operation based on inputs. FSMs appear everywhere in real hardware – traffic controllers, communication protocols, CPU control units. We will design one together, walking through how to identify states, transitions, timing variables, and outputs, before writing the Verilog.

Later in the lab, we briefly touch upon two more powerful Verilog features: **parametric modules**, which let you write a single module that works at any bit-width, and **generate blocks with `genvar`**, which let you instantiate and wire up hardware structures in a loop. **These are not assessed in this lab's graded component but will be useful going forward.**

The labexam is a slightly more advanced problem based on writing combinational logic and FSMs.

## Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'. Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

## Structure of submission/templates

Submission format:

```
your-roll-no/
|- labexam/
    |- jean_grey_decoder.v (TODO)
    |- tb_labexam.v
|- task1/
    |- traffic_fsm.v (TODO)
    |- tb_task1.v
|- task2/
    |- crc.v (TODO)
    |- tb_task2.v
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Alternatively, use the provided `build-archive.sh` script to help you do the same.

```
# cd into the <NN>_inlab folder, NOT the inner folder
$ ./build-archive.sh
```

## Tasks

## Lab Exam: Jean Grey's Secret Channel

Jean Grey is intercepting a serial data stream and injecting **invalid triples** — 3-bit patterns that should never appear in clean data. Your job is to build a clocked sequential circuit that reads the stream **3 bits at a time**, decodes valid triples into 2-bit outputs, and uncovers the secret message Jean hides in her injections.

All resets are **active-high and asynchronous**: use `always @(posedge clk or posedge rst)`.

**Note: In case of any ambiguity the order of resolution is: common clarifications shared by TAs > testbench output and code comments > the problem statement pdf.**

### Encoding Scheme

Only 4 of the 8 possible triples are valid:

| Triple ($ABC$) | Decoded ($XY$) | Valid ($V$) |
|---|---|---|
| 0 0 0 | - - | 0 |
| 0 0 1 | 0 0 | 1 |
| 0 1 0 | 0 1 | 1 |
| 0 1 1 | 1 0 | 1 |
| 1 0 0 | - - | 0 |
| 1 0 1 | 1 1 | 1 |
| 1 1 0 | - - | 0 |
| 1 1 1 | - - | 0 |

The triples `000`, `100`, `110`, `111` are **invalid**. Any invalid triple was placed there by Jean.

### How the System Works

The circuit receives a 4-bit `count` and a `start` signal. When `start` goes high, it reads exactly `count` **valid** triples from the input, one per clock. Each valid triple is decoded and output as a 2-bit `pair`.

When an **invalid triple** is seen:

- It is **ignored** — does not count toward `count`, produces no output.
- The **very next triple** is Jean's secret — it is XORed into a running `secret` accumulator.
- Normal reading then resumes.

When all `count` valid triples are read, `done` is asserted for one cycle.

**Note: Some code will be provided as boilerplate / helper modules to you do NOT edit them.**

### Part (a) — Combinational Decode Logic

**Before writing any Verilog**, derive minimal SOP/POS expressions using Karnaugh maps for:

- **V** — 1 if the triple is valid
- **X** — MSB of decoded output
- **Y** — LSB of decoded output

**Note: For X and Y, invalid triples are don't-cares. This can help you write simpler POS/SOP expressions.**

### Module 1: `triple_decoder`

Purely combinational. **Use only `assign` statements — no `if`, `else`, or `case`. Some reasonable simplification is expected; writing the absolute smallest SOP/POS is not needed.**

```
module triple_decoder (
    input  [2:0] triple,
    output       valid,
    output [1:0] pair
);
    wire A = triple[2];
    wire B = triple[1];
    wire C = triple[0];

    assign valid   = /* V from your K-map */;
    assign pair[1] = /* X from your K-map */;
    assign pair[0] = /* Y from your K-map */;
endmodule
```

### Module 2: `secret_accumulator` (provided – DO NOT MODIFY)

Do not modify this. It XORs the incoming `triple` into `secret` whenever `capture` is high for one posedge.

```
module secret_accumulator (
    input            clk, rst,
    input  [2:0]     triple,
    input            capture,
    output reg [2:0] secret
);
    always @(posedge clk or posedge rst) begin
        if (rst)          secret <= 3'b000;
        else if (capture) secret <= secret ^ triple;
    end
endmodule
```

### Module 3: `jean_grey_decoder`

Top-level module. Instantiates `triple_decoder` and `secret_accumulator`, and contains the FSM.

```
module jean_grey_decoder (
    input            clk, rst, start,
    input  [3:0]     count,
    input  [2:0]     triple,
    output reg [1:0] pair,
    output reg       valid_out,
    output     [2:0] secret,
    output reg       secret_out,
    output reg       done
);
```

**Note:** `secret` is `output` (a wire), wired directly from `secret_accumulator`'s output port — not driven by the FSM.

Declare the following and connect all ports:

```
wire       is_triple_valid;
wire [1:0] triple_pair;
/*reg/wire        capture*/;

triple_decoder td ( /* connect ports */ );
secret_accumulator sa ( /* connect ports */ );
```

**Note: Figure out how to correctly define and set value to `capture` based on intended behaviour.**

### FSM States

```
localparam IDLE      = 2'b00;
localparam READ      = 2'b01;
localparam INTERCEPT = 2'b10;
localparam DONE      = 2'b11;

reg [1:0] state;
reg [3:0] remaining;
```

### Allowed Transitions

```
IDLE      -> IDLE        when start is low
IDLE      -> READ        when start is high; set remaining <- count

READ      -> READ        consume valid triple, when remaining > 1
READ      -> DONE        consume valid triple, when remaining == 1
READ      -> INTERCEPT   consume invalid triple; do NOT decrement remaining

INTERCEPT -> READ        always happens

DONE      -> IDLE        always happens
```

### State Behaviour

**IDLE:** Hold all outputs at 0. Wait for `start`.

**READ:** Check `is_triple_valid` each cycle.

- If valid: drive `pair` and `valid_out`, decrement `remaining`, go to DONE if `remaining == 1`.
- If invalid: go to INTERCEPT — **do not decrement** `remaining`.

**INTERCEPT:** Assert `capture` for exactly this one cycle so `secret_accumulator` XORs the current triple in. Assert `secret_out`. Return to READ.

**DONE:** Assert `done` for one cycle. Return to IDLE.

**Skeleton (IDLE and DONE given) – DO NOT MODIFY**

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        remaining <= 0;
        pair <= 0; valid_out <= 0; done <= 0;
        secret_out <= 0;
    end else begin
        case (state)
            IDLE: begin
                pair <= 0;
                done <= 0; secret_out <= 0; valid_out <= 0;
                if (start) begin remaining <= count; state <= READ; end
            end
            READ:      begin /* your code */ end
            INTERCEPT: begin /* your code */ end
            DONE: begin done <= 1; state <= IDLE; end
        endcase
    end
end
```

## Worked Example

**Please understand output format correctly!**

The table below shows what you would see in the testbench output. Each row is printed **1 ns after a rising clock edge**, so all signals shown are their values **after that clock edge**.

The `state` column is the FSM state after the edge, while `triple` is the input currently present at that time. The testbench uses this same convention when printing its output.

Assume posedge occurs at time 5, 15, etc.

For example: at the rising edge at `t=45`, the module is in `READ` and sees the input `triple=010`. This results in a transition to READ as shown on `t=46` from READ as shown on `t=36`.

```
// Before the rising edge at t=35:
count = 4'b0100; start = 1; triple = 3'b000;
// After the rising edge at t=35:
start = 0;
```

| Time | State | triple | pair | valid_out | secret | secret_out | done |
|---|---|---|---|---|---|---|---|
| 6 | IDLE | 000 | 00 | 0 | 000 | 0 | 0 |
| 16 | IDLE | 000 | 00 | 0 | 000 | 0 | 0 |
| 26 | IDLE | 000 | 00 | 0 | 000 | 0 | 0 |
| 36 | READ | 000 | 00 | 0 | 000 | 0 | 0 |
| 46 | READ | 010 | 01 | 1 | 000 | 0 | 0 |
| 56 | INTERCEPT | 110 | 00 | 0 | 000 | 0 | 0 |
| 66 | READ | 101 | 00 | 0 | 101 | 1 | 0 |
| 76 | READ | 001 | 00 | 1 | 101 | 0 | 0 |
| 86 | READ | 011 | 10 | 1 | 101 | 0 | 0 |
| 96 | DONE | 101 | 11 | 1 | 101 | 0 | 0 |
| 106 | IDLE | 101 | 00 | 0 | 101 | 0 | 1 |

Decoded pairs (valid cycles only): `01 00 10 11`

Jean's secret: `3'b101`

## Task 1: Traffic Light Controller – Help Bhavesh!

Bhavesh wants to cross JVLR road, but traffic is horrible. Help him build a traffic light controller from scratch by implementing three sub-modules and wiring them together in a top-level module.

This is the same system designed in the slides. The goal is to implement it yourself from scratch. Try to write it top-down: wire the three modules together into a working system, then implement each and test.

### Specification

The circuit continuously cycles `GREEN` → `YELLOW` → `RED` → `GREEN`, driven by a 1Hz tick signal. Resetting (`rst_n` active-low) forces the state to `GREEN` with `remaining = 25`.

**State Encodings:** `GREEN` = `2'b00`, `YELLOW` = `2'b01`, `RED` = `2'b10`.

The pedestrian button (`ped_button`) modifies the duration of the current phase and may also be remembered for a later phase using the latched signal `ped_request`.

| State | Normal duration | Duration with request |
|---|---|---|
| GREEN | 25 ticks | Shortened by 10 ticks on button press |
| YELLOW | 5 ticks | 5 ticks |
| RED | 30 ticks | 40 ticks if `ped_request` is latched / `ped_button` is pressed |

A pedestrian button press during `GREEN` immediately shortens the remaining `GREEN` duration by 10 ticks, with a minimum remaining duration of 1 tick, and is remembered. A press during `RED` immediately extends its remaining duration by 10 ticks. A press during `YELLOW` cannot modify the current duration, but is remembered so that the upcoming `RED` phase starts with duration 40 instead of 30.

### Pedestrian Latching

- **Set:** A high `ped_button` pulse sets `ped_request = 1`.
- **Persistence:** The request remains latched after the button is released and is used when entering `RED`.
- **Reset:** `ped_request` is cleared during the `RED` → `GREEN` transition.
- **One-shot modification:** The instantaneous button modifier is enabled only when `ped_button && !ped_request`. This prevents a held button from repeatedly shrinking or extending the duration.

### Understanding `remaining` — Datapath

The `remaining` value (exposed as `duration_out` in the top module) tracks time left in the current state.

- **Initialization:** On reset, `remaining` is initialized to 25.
- **Countdown:** On every `tick_1hz`, if `remaining > 1`, it decrements by 1.

- **Transition:** When `remaining = 1` on a tick, the controller transitions to the next state and loads that state's starting duration.
- **Starting durations:** `GREEN`=25, `YELLOW`=5, and `RED`=30 normally, or 40 when `ped_request=1`.

### Module Structure & Data Flow

Wire three sub-modules inside the top-level `bhavesh_traffic_light`:

- `traffic_fsm` (control/sequential): Owns the registers `current_state`, `remaining`, and `ped_request`. It sequences the states and latches values computed by the datapath.
- `duration_datapath` (combinational): Computes `remaining_next` and `do_transition` from the current state, remaining duration, tick, and pedestrian inputs.
- `output_decoder` (combinational): Takes `current_state` and produces `red`, `green`, and `yellow`.

#### `duration_datapath`

The datapath is purely combinational and owns no registers. Its job is to calculate what should happen to `remaining` before the control logic latches the result.

It first processes the normal 1Hz update. If `tick_1hz` is high and `remaining > 1`, it computes `remaining_next = remaining - 1`. If `remaining = 1`, it asserts `do_transition` and loads the duration of `next_state`.

It then applies the instantaneous pedestrian modifier. If `ped_button && !ped_request && !do_transition` is true, then:

- During `GREEN`: `remaining_next` is reduced by 10, with a minimum value of 1.
- During `RED`: `remaining_next` is increased by 10.
- During `YELLOW`: no immediate duration modification is made; the button is instead latched by the control logic and affects the subsequent `RED` duration.

**Note: The `!do_transition` condition ensures that a pedestrian button present on a transition tick is ignored (refer to slides).**

### Implementation Notes

Fill in the `TODO`s in the provided skeleton file.

### Testbench and Monitor

The testbench runs unit tests on `duration_datapath` and `output_decoder`, followed by full system integration tests. You can read the output to understand what is being tested, and code and comments to see the exact details — you may write your own tests!

Monitor output line format:

```
t=   150 | tick=1 | state=GREEN  remaining= 25 | R=0 G=1 Y=0 | ped=0 | request=0
```

**Testing Tips:**

- View the `.vcd` output for a quick sanity check.
- Reduce durations (e.g. `8'd5`, `8'd3`) while testing to get a much shorter trace.

- Check `[ASSERT]` status lines (`PASS`/`FAIL`) for quick validation.
- Watch `remaining` count down and verify that a transition occurs when it reaches 1 on a tick.
- Confirm that a button press during `GREEN` immediately shrinks the remaining duration by 10, with a floor of 1.
- Confirm that a button press during `YELLOW` is remembered and causes the next `RED` phase to start at 40 ticks.
- Confirm that a button press during `RED` immediately extends the remaining duration by 10.
- Confirm that `ped_request` latches on a button press and clears on the `RED` → `GREEN` transition.
- Remove `clk` from `$monitor` output to reduce the number of output lines.

Note the `make` command will run the testbench and diff with expected output, and print a final `Output matches/does not match` message.:

```
$ make task1
```

## Task 2: CRC Engine

CRC (Cyclic Redundancy Check) is used by real protocols such as Ethernet, USB, and HDMI to verify data integrity. The sender computes a checksum over a stream of bits and appends it to the data. The receiver recomputes the checksum and compares the two values.

You will implement a CRC engine in three parts.

**See the Testing section for how to verify your CRC output against the provided Python script.**

### The Algorithm

CRC is computed serially, one bit at a time. One iteration is:

```
feedback = crc[MSB] XOR incoming_bit

if feedback == 1:
    crc = (crc << 1) XOR POLY
else:
    crc = (crc << 1)
```

Start with `crc = INIT = ff...ff` (all 1s) and feed the message bits through the loop, MSB first. The final `crc` value is the checksum.

**Recall:** For an $N$-bit register, left-shift and append 0 using `{crc[N-2:0], 1'b0}`.

### Implementing CRC-8 (Fixed Width)

Implement the following two CRC-8 modules:

**width = 8 bits, polynomial = `8'hD5`, initial value = `8'hFF`.**

`crc8_update` **(combinational)**

Implement one iteration of the CRC algorithm. The module takes the current CRC value and one input bit, and produces the next CRC value.

Use only `assign` statements.

```
module crc8_update (
    input  [7:0] crc_in,
    input        data_bit,
    output [7:0] crc_out
);
```

`crc8_serial` **(sequential)**

Accumulate the CRC over a stream of bits. On each clock edge where `valid` is high, feed `data_bit` through `crc8_update` and store the result in the CRC register.

Reset the register to `8'hFF` when `rst` is asserted. The `crc` output must always reflect the current register value.

```
module crc8_serial (
    input        clk, rst,
    input        data_bit,
    input        valid,
    input        done,
    output [7:0] crc
);
```

**You must instantiate `crc8_update` inside this module and connect it to the CRC register.**

### Food for thought

Can we extend this algorithm to arbitrary widths of CRC width? What about processing bits in parallel? We will explore these later!

## Testing

### Manual verification

Run the verifier in interactive mode:

```
python3 task2/verifier.py
```

It will prompt you for the message, width, polynomial, and initial value, then print the expected CRC.

### Automated testing

Use the Makefile:

```
$ make task2
```

This compiles your code and produces `task2.vvp`, then runs it once to show debug output. You can open the resulting `task2.vcd` in VaporView.

Finally, it runs the verifier in autograde mode:

```
python3 task2/verifier.py --autograde
```

A successful run will look like:

```
=== RUNNING CRC_VERIFIER.PY ===

Running task2.vvp...

TEST 1: PASS (MSG=AB, WIDTH=8, POLY=D5, INIT=FF, CRC=31)
...

3/3 CRC tests passed.
```